Week 17: Angular + Express Hybrid Application Report

Student Name: Enis Ogdum
Date: January 28, 2026
Course: Dynamic Documents and Front-end Web Development

---

1. Project Overview

This project integrates the Angular password strength evaluator (from Week 16) with an Express.js backend.

- **Frontend (Angular):** Provides the UI and sends the password to the server via HTTP POST.
- **Backend (Express):** Receives the password, logs it, and returns a confirmation message. It also serves the production build of the Angular application.

---

2. Express Backend Source Code

`server.js`

The main entry point for the backend server. It handles the API request and serves static files.

```
const express = require('express');
const cors = require('cors');
const path = require('path');

const app = express();
const PORT = 3000;

// Middleware
app.use(cors()); // Enable CORS for development
app.use(express.json()); // Parse JSON bodies

// API Endpoint
app.post('/api/check-password', (req, res) => {
    const { password } = req.body;

    if (!password) {
        return res.status(400).json({ message: "No password provided" });
    }
```

```
    console.log(`Received password check request for: ${password}`);

    // Send confirmation response
    res.json({
        message: `Password "${password}" was retrieved`
    });
});

// Serve Angular static files (Production)
// Pointing to the specific build output directory
const angularBuildPath = path.join(__dirname, '../frontend/dist/password-
strength-app/browser');
app.use(express.static(angularBuildPath));

// Fallback to index.html for Angular routing
app.get(/^(?!\/api).+/, (req, res) => {
    // Check if index.html exists before trying to send it
    const indexPath = path.join(angularBuildPath, 'index.html');
    res.sendFile(indexPath, (err) => {
        if (err) {
            res.status(404).send("Angular application not built yet. Run 'ng
build' in frontend directory.");
        }
    });
});

// Start server
app.listen(PORT, () => {
    console.log(`Server is running on http://localhost:${PORT}`);
});
```

---

3. Angular Frontend Source Code

src/app/app.config.ts

Configuration file where `HttpClient` is provided.

```
import { ApplicationConfig, provideBrowserGlobalErrorListeners } from
'@angular/core';
import { provideHttpClient } from '@angular/common/http';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideHttpClient(),
  ]
};
```

src/app/password-evaluator/password-evaluator.ts

Component logic handling the HTTP request.

```
import { HttpClient } from '@angular/common/http';
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { CommonModule } from '@angular/common';
import { PasswordEvaluator, PasswordStrength } from '../password-evaluator';

@Component({
  selector: 'app-password-evaluator',
  imports: [FormsModule, CommonModule],
  templateUrl: './password-evaluator.html',
  styleUrl: './password-evaluator.css',
})
export class PasswordEvaluatorComponent {
  password: string = '';
  showPassword: boolean = false;
  strengthInfo: PasswordStrength | null = null;
  serverMessage: string | null = null;
  isLoading: boolean = false;

  constructor(
    private passwordEvaluator: PasswordEvaluator,
    private http: HttpClient
  ) {}

  /**
   * Submit password to backend
   */
  submitPassword(): void {
    if (!this.password) return;

    this.isLoading = true;
    this.serverMessage = null;

    this.http.post<{ message: string }>('http://127.0.0.1:3000/api/check-
password', {
      password: this.password
    }).subscribe({
      next: (response) => {
        this.serverMessage = response.message;
        this.isLoading = false;
      },
      error: (error) => {
        console.error('Error submitting password:', error);
        this.serverMessage = 'Error connecting to server';
        this.isLoading = false;
      }
    });
  }

  /**
   * Toggle password visibility
   */
  togglePasswordVisibility(): void {
    this.showPassword = !this.showPassword;
  }

  /**
```

```
   * Handle password input changes and evaluate strength
   */
  onPasswordChange(): void {
    this.strengthInfo =
this.passwordEvaluator.evaluatePassword(this.password);
  }
}
```

src/app/password-evaluator/password-evaluator.html

Template with the "Submit" button and message display.

```
<div class="password-evaluator-container">
  <h2>Password Strength Evaluator</h2>

  <div class="password-input-wrapper">
    <label for="password-input">Enter Password:</label>
    <div class="input-group">
      <input
        id="password-input"
        [type]="showPassword ? 'text' : 'password'"
        [(ngModel)]="password"
        (input)="onPasswordChange()"
        placeholder="Type your password..."
        class="password-input"
      />
      <button
        type="button"
        class="toggle-button"
        (click)="togglePasswordVisibility()"
        [attr.aria-label]="showPassword ? 'Hide password' : 'Show password'"
      >
        <span class="eye-icon">{{ showPassword ? '👁' : '👁‍🗨' }}</span>
      </button>
    </div>
  </div>

  <div class="strength-indicator" *ngIf="strengthInfo && password.length >
0">
    <div class="strength-label-wrapper">
      <span class="strength-label-text">Strength:</span>
      <span
        class="strength-label"
        [style.color]="strengthInfo.color"
        [style.border-color]="strengthInfo.color"
      >
        {{ strengthInfo.label }}
      </span>
    </div>

    <div class="strength-bar-container">
      <div
        class="strength-bar"
        [style.width]="(strengthInfo.score / 4 * 100) + '%'"
        [style.background-color]="strengthInfo.color"
      ></div>
```

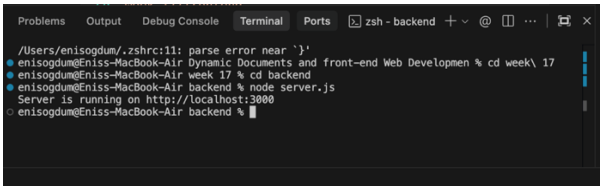
```
    </div>

    <div class="suggestions" *ngIf="strengthInfo.suggestions.length > 0">
      <h4>Suggestions:</h4>
      <ul>
        <li *ngFor="let suggestion of strengthInfo.suggestions">{{ suggestion
}}</li>
      </ul>
    </div>
  </div>

  <div class="submit-section">
    <button
      class="submit-button"
      (click)="submitPassword()"
      [disabled]="!password || isLoading"
    >
      {{ isLoading ? 'Sending...' : 'Submit to Server' }}
    </button>
  </div>

  <div class="server-message" *ngIf="serverMessage">
    <p>{{ serverMessage }}</p>
  </div>
</div>
```

---

4. Screenshots

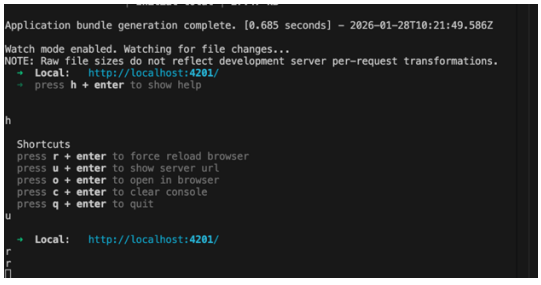
Backend Terminal

Shows the Express server running on port 3000.



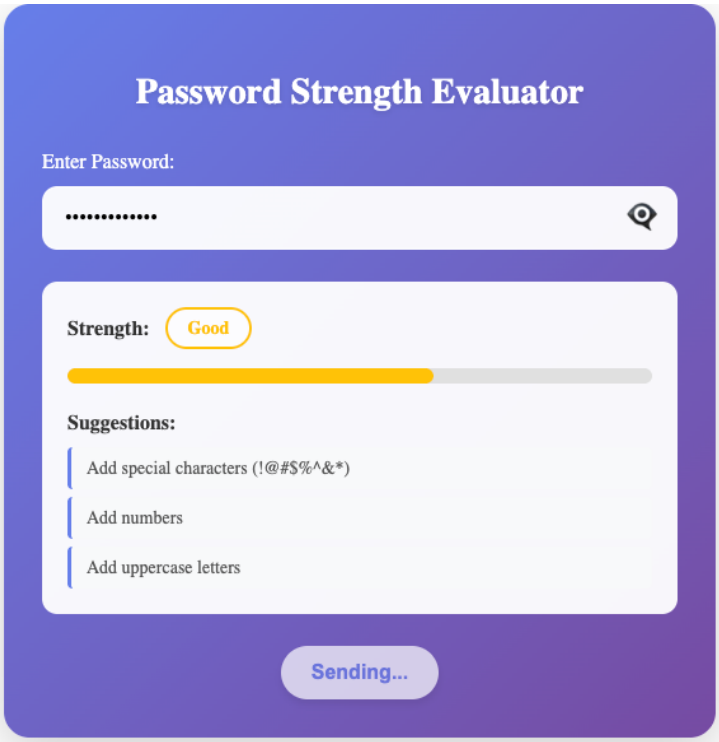
Frontend Terminal

Shows the Angular development server running on port 4201.



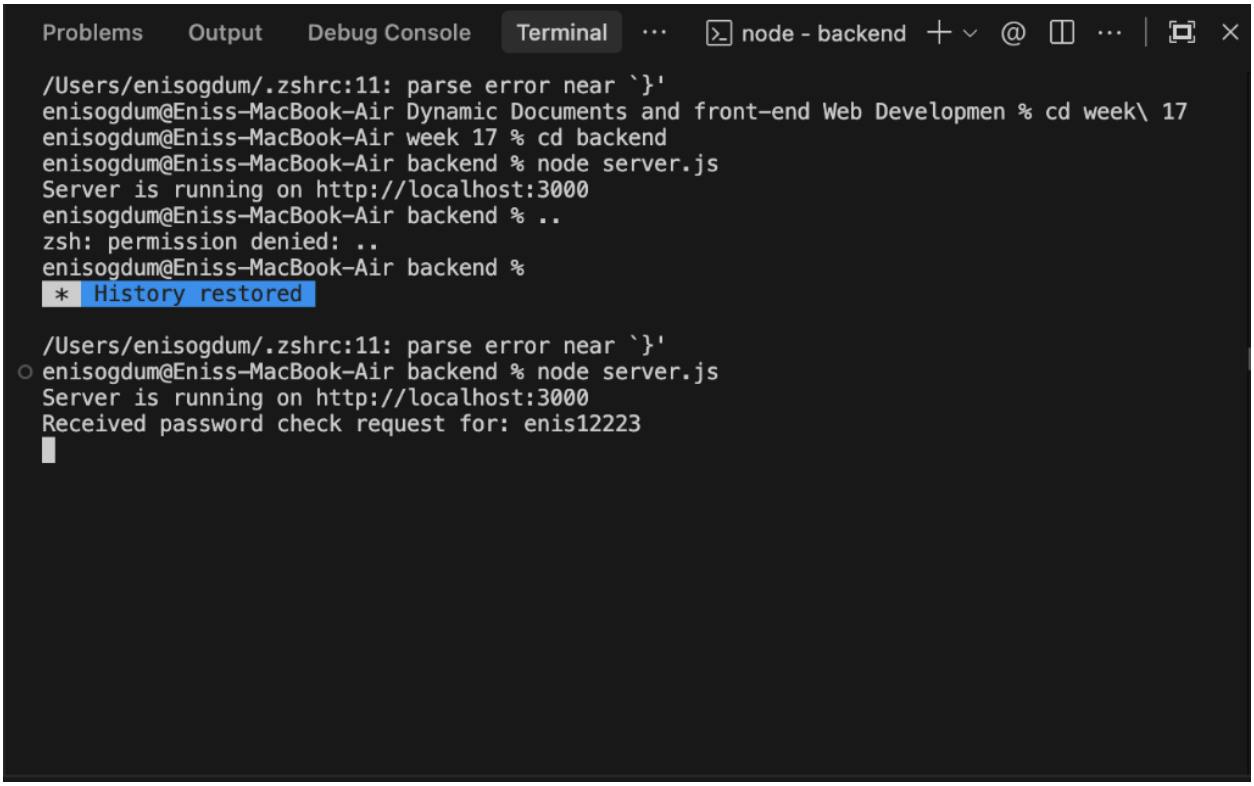
Application State (Sending Request)

Shows the application with a password entered ("Good" strength) and the state after clicking submit ("Sending…").



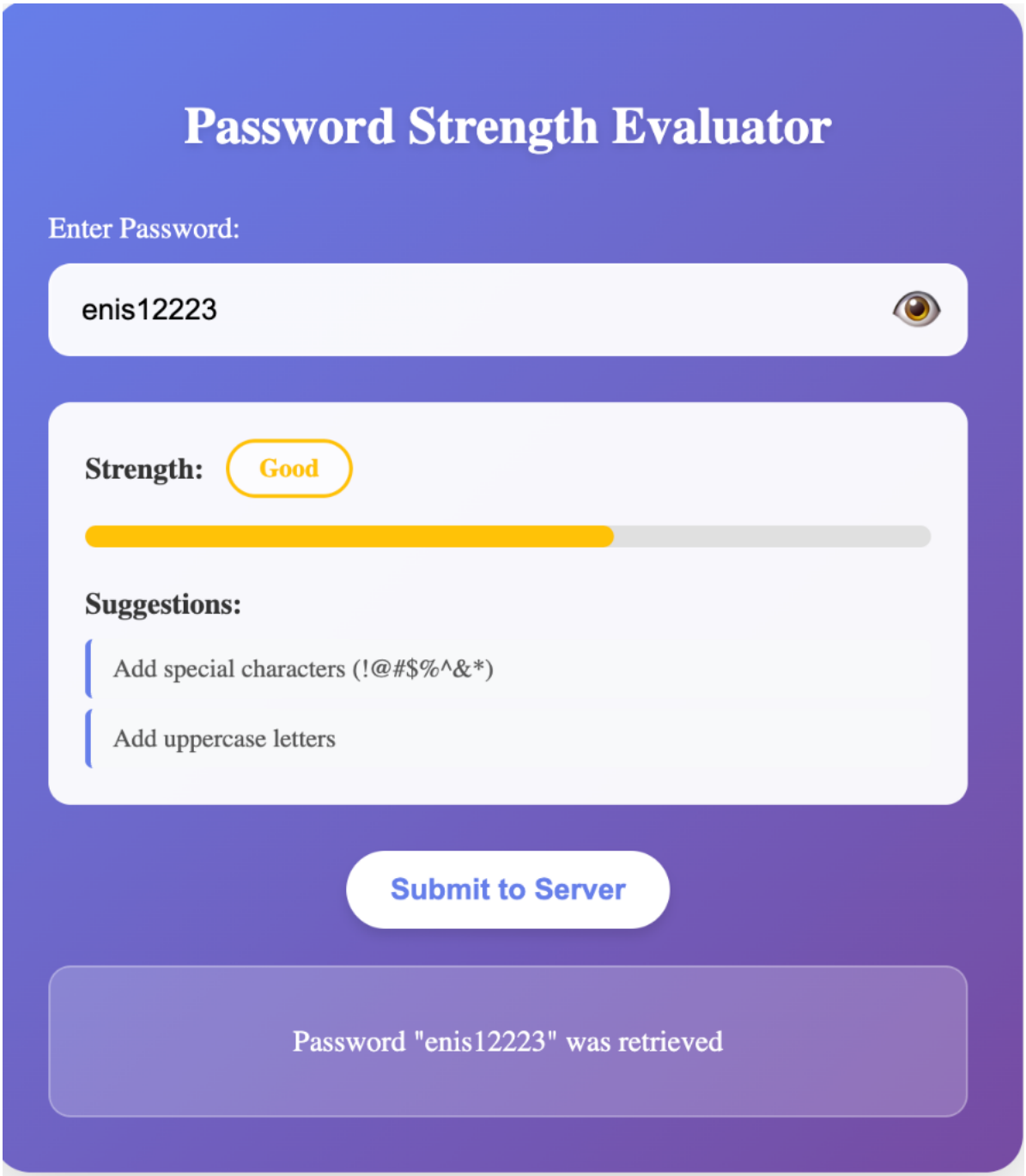
Backend Reception

Shows the Express server console output when the password is received.



Frontend Confirmation

Shows the application UI after successfully receiving a response from the server, displaying the retrieval message.



(Note: Upon successful completion, the application displays "Password [XYZ] was retrieved" in the area below the button.)

End of Report